

Anforderungen an Synchronisations-Mechanismen:

Gegenseitiger Ausschluss (Nur ein Thread darf in Critical Section sein), Fortschritt (Entscheidung, wer in die Critical Section darf, muss zu endlich Zeit getroffen werden), Begrenztes Warten (Thread wird nur n mal übergeben, bevor er in die Critical Section darf).

Implementierung: Nur mit HW-Unterstützung möglich. Es gibt zwei atomare Instruktionen:

Test-And-Set (Setzt einen int auf 1 und returt den vorherigen Wert: test_and_set(int & target) { int value = *target; *target = 1; return value; } und Compare-and-Swap (Überschreibt einen int wenn ein spezifiziertes Wert, wenn dieser dem erwarteten Wert entspricht: compare_and_swap (int & int expected, int & int value, int & int value) { if (value == expected) { *a = value; } return value; }).

SEMAPHORE

Initialer Zähler z ≥ 0. Wird nur über Post(v) (Erhöhe z um 1) und Wait(v) zugegriffen (Wart: Wenn z > 0, verringert z um 1 und fährt fort, wenn z = 0, setzt den Thread in den Status, bis anderer Thread z erhöht).

Alle diese Funktionen geben, gemäss POSIX API (Thread Library), den Status eines zueits, zurückgibt.

sem_init (&sem_t &sem, int shared, unsigned int value); Initialisiert Semaphore vom Typ sem_t, sem_t muss als globale Variable oder member von struct, pthread_t oder einem Prozessor, value=Startwert von Semaphore.

Verwendung globaler Semaphore über mehrere Prozesse:

sem_t sem; /*global*/ int main (...) { sem_init (&sem, 0, 4); /* pthreaded, Semaphore Wert auf 4 */ sem_wait (&sem_t &sem); /* sem_wait (&sem_t &sem); // implementieren Port & Wait - sem_trywait (&sem_t &sem); sem_timedwait (&sem_t &sem, const struct timespec *abs_timeout); Sind wie sem_wait, aber brechen ab, falls Dekrement nicht durchgeführt werden kann, sem_trywait bricht sofort ab, sem_timedwait nach der angegebenen Zeitdauer. sem_destroy (&sem_t &sem); Entfernt Speicher, den das OS mit sem assoziiert hat.

semaphore free = n; semaphore used = 0; // Beispiel mit 1 Producer und 1 Consumer

while (C) { // Consumer while (C) { // C WAIT (free); //warte auf Platz in Queue produce_item (C,waitPlatz, ...); WAIT (used); // Hat es Elem. in Queue? POST (used); // 1 Element mehr in Queue consume (&buffer[S]); // Element entfernt, wenn in Q = n - (wait) < BUFFER_SIZE; POST (free); // Element freigegeben, wenn in Q = n - (wait) < BUFFER_SIZE; } } }

MUTEXE

Ein Mutex hat einen binären Zustand z, der nur durch zwei Funktionen verändert werden kann: Acquire (Wenn z = 0, setzt z auf 1 und fährt fort, Wenn z = 1, blockiert den Thread, bis z = 0), Release (Setzt z = 0).

Auch als non-blocking-Funktion: pthread_mutex_trylock (pthread_mutex_t *mutex); // Beispiel Initialisierung pthread_mutex_t mutex; // global int main() { pthread_mutex_init (&mutex, 0); // Enter critical section: pthread_mutex_lock (&mutex); // run threads & wait for them to finish pthread_mutex_unlock (&mutex); // Leave critical section: pthread_mutex_unlock (&mutex); ... }

Priority Inversion: Ein hoch-priorisierter Thread C wartet auf eine Ressource, die von einem niedriger priorisierten Thread A gehalten wird. Ein Thread mit Priorität zwischen diesen beiden Threads erhält den Prozessor. Kann mit Priority Inheritance gelöst werden. Die Priorität von A wird temporär auf die Priorität von C gesetzt, damit der Mutex schnell wieder freigegeben wird.

SEMPHORE, PIPES UND SOCKETS

SIGNAL

Signale ermöglichen es, einen Prozess von aussen zu unterbrechen, wie ein Interrupt. Unterbrechen des gerade laufenden Prozesses/Threads, Auswahl und Ausführen der Signal-Handler-Funktionen, Fortsetzen des Prozesses. Werden über ungültige Instruktionen oder Abbruch auf Seitens Benutzer ausgelöst. Jeder Prozess hat pro Signal einen Handler.

Mit kill kann ein Signal, mit einem dieser Default-Handler, gesendet werden:

Abnormal-Terminate-Handler | Beendet Programm + Core Dump (für Programmfehler) | SIGSEGV (fehler in arithmetischen Operationen), SIGILL (linguistische Instruktionen)

Terminate-Handler | Beendet das Programm (Prozess abbrechen) | SIGKILL (nicht übersehbar), SIGTERM (Normale höfliche Anfrage an den Prozess, sich zu beenden), SIGINT (Nichtakzeptierte Aufforderung an Prozess, sich zu beenden), SIGQUIT (Wie SIGINT, aber anomale Terminierung Ctrl-\), SIGABRT (Wie SIGQUIT, aber vom Prozess an sich selber (Programmierfehler bemerkli),

Ignore-Handler | Ignoriert das Signal | SIGSTPPT (Versteht den Prozess in den Zustand stopped, ähnlich wie waiting Ctrl-Z), SIGSTOP (Wie SIGSTP, aber nicht übersehbar), SIGCONT (Setzt den Prozess fort (fg))

int sigaction(int signal, struct sigaction *new, struct sigaction *old);

Die Signal-Handler für signal, wenn new = 0, (Signal-Signal-Handler definiert was signal struct: sa_handler; Zu callende Funktion, sa_mask: Blockierte Signale während Ausführung, bearbeitet nur durch signal(2) (Signalname: signaltypname, sigfillset, sigdempty, sigdelset, sigismember)

sigaction = transp. Datenstruktur, signal_t = opake Datenstr. + implementationsabhängig

PIPES

geöffnete Datei: entspricht einem Eintrag in der File-Descriptor-Tabelle (FD) im Prozess. Zugriff über File-API (open, close, read, write,...). Das OS speichert je Eintrag der Prozess-FD ein Verweis auf die globale FD.

fork(0); die FD wird hier kopiert. int dup (int source_fd, int destintion_fd); Duplizieren und returen den FD source_fd, dup aliziert neuen FD, dup2 überschreibt destination_fd.

Umschleiten des Ausgabestreams

int fd = open("log.txt", ...); int pid = fork(0); if (pid == 0) { // child dup2(fd, 2); // duplicate fd for log.txt as standard output // e.g., load new image with execv, fd's remain // else { // parent <= close (fd); }

(Eine Datei muss nur opened & close & read/write unterstützen)

Pipe: >Dateien im Hauptspeicher, die über zwei FD verwendet wird: read end zum Lesen, write end zum Schreiben in die Pipe kein Leck.

Daten, die in write end geschrieben werden, können aus read end genau einmal gemäss FIFO gelesen werden. Pipes erlauben Kommunikation über Prozess-Grenzen hinweg. In unidirektional int pipe (int fd [2]); // equivalent to int pipe (int *fd)

if (id == 0) { // Child thread pipe (fd); pipe (fd [1]); // don't use write end char buffer [BSIZE]; while (1) { // read (fd[0], &buffer, BSIZE); // else { // Parent thread pipe (fd[1]); // don't use read end char * text = "I <3 str&fart(1); write (fd[1], text, strlen(text) + 1); }

unklar, wer die Daten erhält.

– keine Daten in der Pipe -> read blockiert, bis Daten hineingeschrieben werden

– keine Daten in der Pipe + kein geöffnetes Write-End -> read gibt 0 zurück (EOF)

– Default Pipe Größe = 16 Pages, meistens 64KB

int mkfifo (const char *path, mode_t mode); Erzeugt eine Pipe mit Namen und Pfad im Dateisystem. Hat via mode Permission Bits wie normale Datei. Wird unabhängig vom erzeugenden Prozess, je nach System auch über Reboots hinweg. Muss explizit mit unlink gelöscht werden.

SOCKETS

Ein Socket repräsentiert einen Endpunkt auf einer Maschine. Kommunikation findet im Regelfall zwischen zwei Sockets statt (JUDG TCP über IP sowie Unix Domain Sockets).

Sockets benötigen für Kommunikation einen Namen: (IP-Adresse & Portnummer)

int socket(int domain, int type, int protocol); Erzeugt einen neuen Socket als >Daten. Socket sind nach Erzeugung zunächst unbenannt. Alle Operationen blockieren per default.

Domain (AF_INET, AF_INET6, type (SOCK_STREAM, SOCK_DGRAM, SOCK_STREAM, protocol (System-specific, 0 = Default-Protocol)

Client: connect (Verbindung unter Angabe einer Adresse aufrufen); listen (Senden von Daten, 0 -> n, recv (Empfangen von Daten, 0 -> n, close (Schliessen der Verbindung)

Server: bind (festlegen einer nach aussen sichtbaren Adresse), listen (Bereitstellen einer Queue zum Sammeln von Verbindungsanfragen von Clients), accept (Erzeugen einer Verbindung auf Anfrage von Client), recv (Empfangen von Daten, 0 -> n, send (write (Senden von Daten, 0 -> n, close (Schliessen der Verbindung)

struct sockaddr_in ip_addr; ip_addr.sin_port = htons (443); // default HTTPS port inetpton (AF_INET, "192.168.0.1", &ip_addr.sin_addr.s_addr); // IP address in memory: 0x00 0x0A 0x00 0x01, Port in memory: 0x01 0x0B

htons() konvertiert 16 Bit von Host-Byte-Order in Network-Byte-Order, htons() 32 Bit. ntohs() und ntohl() sind Gegenstücke. inetpton() konvertiert protokol-spezifische Adresse von String zu Network-8. inet_ntop() ist das Gegenstück (network-to-presentation).

int bind (int socket, const struct sockaddr *local_address, socklen_t addrlen); bindt den Socket an die angegebene, unbenutzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis der Vorgang abgeschlossen ist.

int connect (int socket, struct sockaddr *remote_address, socklen_t addrlen); Aufbau einer Verbindung. Bindet den Socket an eine neue, unbenutzte lokale Adresse, wenn noch nicht gebunden. Blockiert, bis Verbindung steht oder ein Timeout eintritt.

int listen (int socket, backlog); Markiert den Socket als >bereit zum Empfang von Verbindungen. Erzeugt eine Warteschlange, die so viele Verbindungsanfragen aufnehmen kann, wie blocklog abgibt.

int accept (int socket, struct sockaddr *remote_address, socklen_t addrlen); Wartet, bis Verbindungsanfrage in der Warteschlange eintrifft. Erzeugt einen neuen Socket und bindet ihn an eine neue lokale Adresse. Die Adresse des Clients wird in remote_address geschrieben. Der neue Socket kann keine weiteren Verbindungen annehmen, der bestehende schon.

Typisches Muster für Server

int server_fd = socket (...); bind (server_fd, ...); listen (server_fd, ...); while (running) { int client_fd = accept (server_fd, 0, 0); delegate_to_worker_thread (client_fd); // will call close(client_fd)

size_t send (int socket, const void *buffer, size_t length, int flags); // wie write: // size_t write (int fd, void *buffer, size_t n) size_t recv (int socket, void *buffer, size_t length, int flags); // wie read: // size_t read (int fd, void *buffer, size_t n) send (fd, buf, len, 0); // Aufruf Beispiel

Senden und Empfangen von Daten

Erweitern read(0) bzw. write(0) um Socket-Funktionalitäten durch den flags-Parameter. Puffern der Daten ist Aufgabe des Netzwerkstacks.

int close (int socket); Schliesst Socket für den aufrufenden Prozess. Hat anderer Prozess den Socket geöffnet, bleibt Verbindung bestehen. Gegenseite wird nicht benachrichtigt. Gegenseite erhält beim read kein EOF und verlässt die Lese-Schleife nicht.

int shutdown (int socket, mode); Schliesst Socket für alle Prozesse und baut die entsprechende Verbindung ab. mode: SHUT_RD (keine Lese-Zugriffe mehr), SHUT_WR (keine Schreib-Zugriffe mehr), SHUT_RDWR (keine Lese- oder Schreib-Zugriffe mehr)

MESSAGE PASSING UND SHARED MEMORY UND PIPES

Prozesse sind voneinander isoliert, müssen jedoch trotzdem miteinander interagieren.

Shared Memory ist schneller zu realisieren, aber schwer wartbar. Message-Passing erfordert mehr Engineering-Aufwand, schlussendlich aber in Mehr-Prozessor-Systemen bald performanter.

MESSAGE-PASSING

int Mechanismus mit zwei Operationen: – Send (Kopiert die Nachricht aus dem Prozess: send (message)), – Receive (Kopiert die Nachricht in den Prozess: receive (message)).

Unterscheidungskriterien

Dabei können Implementierungen nach verschiedenen Kriterien unterschieden werden

Feste oder variable Nachrichtengröße: feste Nachrichtengröße ist einfacher zu implementieren, aber umständlicher zu verwenden als variable Nachrichtengröße.

Direkte Kommunikation: Kommunikation nur zwischen genau zwei Prozessen, Sender muss Empfänger kennen. Es gibt symmetrische direkte Kommunikation (Empfänger muss Sender kennen, können jedoch die ID in einem Call-Parameter mitteilen (recv_id, message)).

Asymmetrische direkte Kommunikation (Empfänger muss Sender nicht kennen, sondern erhält die ID in einem Call-Parameter mitteilen (recv_id, message)).

Indirekte Kommunikation: Prozess sendet Nachricht an Mailboxen, Ports oder Queues. Empfänger empfängt aus diesem Objekt. Beide Teilnehmer müssen die gleiche(n) Mailbox(en) kennen.

Weitere: synchrone oder asynchrone Kommunikation, Pufferung, mit oder ohne Prioritäten für Nachrichten

Indirekte Kommunikation

Lebenszyklus-Queue: Wenn diese Queue einem Prozess gehört, lebt sie solange wie der Prozess. Wenn sie dem OS gehört, muss das OS das Löschen übernehmen.

Synchronisation: Blockierendes Senden (Sender wird solange blockiert, bis die Nachricht vom Empfänger empfangen wurde), Nicht-blockierendes Senden (Sender sendet Nachricht und fährt sofort weiter), Blockierendes empfangen (Empfänger wird blockiert, bis Nachricht verfügbar), Nicht-blockierendes empfangen (Empfänger erhält Nachricht, wenn verfügbar, aber nicht).

Rendezvous: Sind Sender und Versand beide blockiert, kommt es zum Rendezvous, sobald beide Seiten ihren Aufruf getätigt haben. Impliziter Synchronisationsmechanismus.

Pufferung: Keine (Queue-Länge ist 0, Sender muss blockieren), Beschränkte (Maximal n Nachrichten, Sender blockiert, wenn Queue voll ist), Unbeschränkte (Beliebig viele Nachrichten, Sender blockiert nie).

Prioritäten: In manchen Systemen können Nachrichten mit Prioritäten versehen werden. Der Empfänger holt die Nachricht mit der höchsten Priorität zuerst aus der Queue.

POSIX Message-Passing = OS Message Queue API

Nutzt OS Message-Queues mit variabler Länge, haben mind. 32 Prioritäten und können synchron und asynchron verwendet werden, indirekte Kommunikation.

int mq_open (const char *path, int flags, mode_t mode, struct mq_attr *attr); Öffnet eine Message-Queue mit systemtemischem name, return Message-Queue-Descriptor. (name mit *a beginnt, flags & mode wie bei Dateien, mq_attr: Ds. Konfig & Queue-Status, RW mit get_attr/mask/set_attr)

int mq_close (mqd_t mq); Schliesst die Queue mit dem Descriptor queue für diesen Prozess.

int mq_unlink (const char *name); Entfernt die Queue mit dem Namen name aus dem System. Name wird sofort entfernt und es kann kein anschießend nicht mehr geöffnet werden.

int mq_send (mqd_t queue, const char *msg, size_t length, unsigned int priority); Sendet die Nachricht, die an Adresse msg beginnt und length Bytes lang ist, in die queue.

int mq_receive (mqd_t queue, const char *msg, size_t length, unsigned int *priority); Kopiert die nächste Nachricht aus der Queue in den Puffer, der an Adresse msg beginnt und length Bytes lang ist. Blockiert, wenn die Queue leer ist.

Message-Queues

– bidirektional

– Daten sind in einzelnen Messages organisiert

– beliebiger Zugriff

– Haben immer einen Namen

Pipes

– unidirektional

– übermittelnl Bytestrom an Daten

– beliebiger Zugriff

– Müssen keinen Namen haben

SHARED MEMORY

Frames des Hauptspeichers werden zwei (oder mehr) Prozessen P1 und P2 zugänglich gemacht. In P1 wird Page V1 auf einen Frame F abgebildet. In P2 wird Page V2 auf denselben Frame F abgebildet. Beide Prozesse können beliebig auf dieselben Daten zugreifen. Im Shared Memory müssen relative Adressen verwendet werden.

POSIX API

Das OS benötigt eine >Datei S, das Informationen über den gemeinsamen Speicher verwaltet und eine Mapping Table je Prozess.

POSIX (mode für shm, open: S_IRWXU = 0700, S_IRUSR = 0200, S_IWUSR = 0040, S_IRGRP = 0001, S_IWGRP = 1 verknüpft.

shm_open (C"/var/ahemshare", O_RDWR | O_CREAT, S_IRUSR | S_IWUSR); Erzeugt (falls nicht) einen offener Shared Memory /var/ahemshare zum Lesen und Schreiben.

int shmctl (int shm, int cmd, off_t *length); Setzt Größe der >Datei. Muss zwingend nach SM-Erstellung gesetzt werden, um entsprechend viele Frames zu allozieren.

Für Shared Memory mit ganzzahligen Vielfachen der Frame-/Framegröße verwendet.

int shm_unlink (const char *name); Löscht das Shared Memory mit dem name. (bleibt vorhanden, solange noch von Prozess genutzt)

int shmmap (void *address, size_t length); Entfernt das Mapping.

void *address = mmap(0 // maps shared memory into virt. address space of process 0 // void *init_address (0 because nobody cares) size_of_shared_memory // size_t length (same as used in shmctl)

PROT_READ | PROT_WRITE, // int protection (never use execute) MAP_SHARED, // int flags fd, // int file descriptor, // off_t offset (start map from first byte)

UNICODE

ASCII - AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE

Hex 10101A, 1011-8, 1100-8, 1101-0, 1110-E, 1111-F

Hat 128 mögliche Zeichen (erste Hexzahl = Zeile v., zweite Hexzahl = Spalte -> d.h. a1 = A).

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	ML	DEL	1	2	3	4	5	6	7	8	9	A	B	C	D	E
1	ML	DEL	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
3	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
4	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	]	^	_	`
6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	}	~		

Codepages: unabhängige Erweiterungen auf 8 Bit. Jede ist unterschiedlich und nicht erkennbar.

Unicode: Hat zum Ziel, einen eindeutigen Code für jedes vorhandene Zeichen zu definieren. D8,00, bis DF FF, sind wegen UTF-16 keine gültigen Code-Points.

Code-Points (CP oder P): Nummer eines Zeichens - >welches Zeichen? in der Unicode Tabelle

Code-Unit (CU): Einheit, um Zeichen in einem Encoding darzustellen (bietet den Speicherplatz)

P1 = 1es Bit des uncodierten CPs, U1 = 1es Code-Unit des kodierten CPs, U1 = 1es Byte des kodierten CPs

UTF-32

Jede CU ist die ganze 32 Bit gross, jeder CP ist eine CU. Wird häufig intern in Programmen verwendet. Obere 11 Bits oft >zweckentfremdet. Endianess gibt an, mit welchem Ende die Folge anfangt.

Big-Endian: 83 B2 B1 B0 0 aus Unicode-Tabelle, Little-Endian: 80 B1 B2 B3

UTF-8

Jede CU umfasst 8 Bit, ein CP benötigt 1 bis 4 CUs. Standard für Webpages. Echte Erweiterung von ASCII. Jedes B (Byte) = 1 Code-Unit, Maximal 4 Bytes.

Code-Point in U1 U2 U3 U4 signifikant

00...7FF, 80...7FF, 800...FFFF, 10000...10FFFF, 11110xxx, 10xxx xxxx, 10xxx xxxx, 10xxx xxxx

7 bits, 11 bits, 16 bits, 21 bits

Vorgehen UTF-8

1. P = E4, = 1. in Binär umwandeln 1110 0100

2. von rechter Seite her die Bits aufteilen und Separator-Bits korrekt einfügen:

1 | 11010010 -> 110 01001 10 100100

3. am Schluss zurück in Hex umwandeln: C3 44

UTF-16

Jede CU umfasst 16 Bit, ein CP benötigt 1 oder 2 CUs. Die 2 CUs werden Surrogate Pair genannt, U1: high surrogate, U2: low surrogate. Bei 2 Bytes (1 cu) wird Kombination mit vorne mit Nullen angereiht, CP in 0-FFFF hoch - 0FFF. Bei 4 Bytes sind D8,00, bis DF,FF (Bis 1272) wegen dem Separator ungültig und müssen umgerechnet werden.

BE: U0 = B1 B0, LE: U1 U0 = B3 B2 B1 B0, U1 U0 = B2 B3 B1 B0

–1'0000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000, 00000000

00000000, 00000000, 00000000, 000